
MANGOBOOST

# AMDSOW InferenceX project detail

## Config, workflow, Slurm, Docker, and vLLM wiring

AMDSOW InferenceX DeepSeek-R1-0528 FP8 MI300X vLLM PD-disaggregation validation package

Confidential. Shared with AMD under NDA. | contact@mangoboost.io

## Document control

| Field | Value |
| --- | --- |
| Document | AMDSOW InferenceX project detail |
| Version | v1.0.0 |
| Last updated | 2026-06-25 |
| Owner | MangoBoost AMDSOW delivery team |
| Contact | contact@mangoboost.io |
| Audience | Maintainers, delivery owners, and package reviewers |
| Status | Generated A4 PDF from maintained AMDSOW InferenceX Markdown documentation |
| Source format | Maintained Markdown rendered through HTML/CSS with Python-Markdown and WeasyPrint. |

Table of contents



# AMDSOW InferenceX project description and detail

This document explains how the AMDSOW DeepSeek-R1-0528 FP8 MI300X vLLM PD-disaggregation benchmark is wired together in this repo: which files own which decisions, how a YAML row turns into a running benchmark, what the key environment variables mean, and what is safe to change.

It is the reference companion to `docs/AMDSOW_INFERENCEX_USER_GUIDE.md`. The user guide has the copy/paste commands for running and collecting a benchmark. This document does not repeat those commands. Read it when you need to understand the structure or edit the config or scripts.

Everything here describes the repository revision that carries this document. Keep the script and config changes in the same commit as this doc; otherwise the code excerpts and behavior notes will drift.

## 1. Purpose

InferenceX is a benchmark harness. A single YAML file describes benchmark recipes. GitHub Actions expands the YAML into jobs, a self-hosted runner submits a Slurm job, Slurm allocates MI300X nodes, Docker containers start vLLM in a prefill/decode split, a benchmark client drives traffic, and the run uploads JSON results.

This document focuses on one config key, `dsr1-fp8-mi300x-vllm-disagg`. That key serves DeepSeek-R1-0528 in FP8 on AMD MI300X nodes using vLLM with prefill/decode (PD) disaggregation. The same

workflow and runner machinery also drives other keys (SGLang, ATOM, other SKUs), but the disaggregated MI300X path has its own recipe and runtime scripts and is what this document covers in detail.

PD disaggregation splits inference into two server roles:

| Role | Work | KV role |
|---|---|---|
| Prefill | Reads the input prompt and builds the KV cache. | `kv_producer` |
| Decode | Generates output tokens. | `kv_consumer` |

The prefill servers transfer KV cache to the decode servers over RDMA using the MoRIIO connector. A proxy or router on the first prefill node fronts both roles behind one HTTP port.

# 2. Source-of-truth files

These files own the behavior. When the docs and the files disagree, the files win.

| File | Role |
|---|---|
| `.github/configs/amd-master.yaml` | The benchmark matrix. The `dsr1-fp8-mi300x-vllm-disagg` key defines the model, image, runner type, and the prefill/decode search space. |
| `.github/configs/runners.yaml` | Maps a runner type (for example `mi300x-disagg`) to the concrete self-hosted runner names that can accept the job. |
| `utils/matrix_logic/generate_sweep_configs.py` | Expands the YAML into a JSON matrix. The `test-config` and `full-sweep` subcommands build the per-job entries. |
| `utils/matrix_logic/validation.py` | Loads and validates `amd-master.yaml` and `runners.yaml` (`load_config_files`, `load_runner_file`) and validates each matrix entry. |
| `.github/workflows/e2e-tests.yml` | Entry workflow. Runs the generator, splits the matrix into single-node, multi-node, agentic, and eval groups, and fans out the template jobs. |
| `.github/workflows/benchmark-multinode-tmpl.yml` | The multi-node job. Sets `runs-on` to the runner type, exports the per-row env, and calls the launcher. |
| `.github/workflows/collect-results.yml` | Aggregates per-job benchmark artifacts into `agg_bmk.json`. |
| `runners/launch_mi300x-amds.sh` | The MI300X launcher. For the multi-node path it sets cluster env (partition, model dir, RDMA devices), derives the recipe name, submits it, tails the Slurm log, and copies result files back to the workspace. |
| `benchmarks/multi_node/dsr1_fp8_mi300x_vllm-disagg.sh` | The recipe. Turns the exported prefill/decode env into the booleans and flags the rest of the chain expects, then calls `submit.sh`. |
| `benchmarks/multi_node/amd_utils/submit.sh` | Builds and submits the `sbatch` command. Computes node count and per-worker TP size, and forwards every variable to the job via `--export=ALL`. |
| `benchmarks/multi_node/amd_utils/job.slurm` | The Slurm batch script. Validates the model, selects nodes, builds Docker env, fans out one container per node with `srun`, and starts the router/proxy container on rank 0. |
| `benchmarks/multi_node/amd_utils/server.sh` | Container entrypoint dispatcher. Sources `server_vllm.sh` for `vllm-disagg`. |
| `benchmarks/multi_node/amd_utils/server_vllm.sh` | The vLLM launcher. Assigns a role from `NODE_RANK`, assembles `vllm serve` flags, starts the server, and on rank 0 runs the benchmark and optional eval. |
| `benchmarks/multi_node/amd_utils/models_vllm.yaml` | Model-specific vLLM flags and env. The `DeepSeek-R1-0528` entry holds the TP8 profile and the separate DP8EP profile. |
| `benchmarks/multi_node/amd_utils/env.sh` | Shared per-node RDMA and NCCL/RCCL environment. Sourced by `server_vllm.sh`. |

| File | Role |
|---|---|
| `benchmarks/multi_node/amd_utils/bench.sh` | The benchmark client wrapper. Runs `benchmark_serving.py` once per concurrency and writes the per-concurrency JSON. |
| `benchmarks/multi_node/amd_utils/sync.py` | Barrier and port-wait helper used between the prefill, decode, and proxy processes. |
| `benchmarks/benchmark_lib.sh` | Shared shell helpers ( `check_env_vars` , `run_benchmark_serving` , `run_eval` ). |
| `utils/process_result.py` | Per-file result processing into `agg_<RESULT_FILENAME>_*.json` . |
| `utils/calc_success_rate.py` | Builds `run_stats.json` from the collected results. |

# 3. Runner, node, and worker are three different things

These three terms are easy to confuse. They refer to different layers and are set in different places.

GitHub runner type (also called the runner label). A string like `mi300x-disagg` . It comes from the `runner:` field of the config key in `amd-master.yaml` . The multi-node template uses it as `runs-on` , so it selects which pool of self-hosted runners can pick up the job. `runners.yaml` lists the concrete runner names behind each type:

Code block 1: `.github/configs/runners.yaml` lines 107-110.

```
mi300x-disagg:
- 'mi300x-amds_06'
- 'mi300x-amds_07'
- 'mi300x-amds_08'
```

GitHub runner name. The registered name of the specific machine that accepted the job, for example `mi300x-amds_06` . The workflow passes it as `RUNNER_NAME` and uses it two ways:

- The launcher script is chosen by stripping at the first underscore: `bash ./runners/launch_${RUNNER_NAME%%_*}.sh` resolves to `runners/launch_mi300x-amds.sh` .
- It becomes the Slurm job name. The workflow cancels stale jobs by that name before a run, and the launcher cancels the submitted `JOB_ID` at the end.

Slurm compute node. A physical MI300X host with 8 GPUs that Slurm allocates for the job. The node count per job is `PREFILL_NODES + DECODE_NODES` . These nodes are not named in the repo and must be discovered at run time.

Benchmark worker. One vLLM server process in a role. `xP` prefill workers and `yD` decode workers, where `xP = PREFILL_NUM_WORKERS` and `yD = DECODE_NUM_WORKERS` . `server_vllm.sh` assigns exactly one role per node rank, so the harness runs one worker per node.

# 4. Config matrix semantics

## 4.1 The config key

The `dsr1-fp8-mi300x-vllm-disagg` key in `amd-master.yaml` sets the run-wide fields:

Code block 2: `.github/configs/amd-master.yaml` lines 169-181, trimmed excerpt.

```
dsr1-fp8-mi300x-vllm-disagg:
  image: docker.io/chaeminlimmb/vllm-mori-pd:milestone4-aiterwheel
  model: deepseek-ai/DeepSeek-R1-0528
  model-prefix: dsr1
  runner: mi300x-disagg
  precision: fp8
  framework: vllm-disagg
  multinode: true
  disagg: true
  scenarios:
```

```
    fixed-seq-len:
    - isl: 1024
      osl: 1024
      search-space:
      - ...
```

Under `scenarios.fixed-seq-len` there are two sequence-length blocks (1k1k and 8k1k). Each block has a `search-space` list. Each search-space entry describes one prefill/decode topology and carries a concurrency list.

## 4.2 How the generator expands a search-space entry

For a multi-node config, `generate_sweep_configs.py` produces one matrix entry per search-space entry and sets `conc` to the entire `conc-list`. It does not produce one entry per concurrency. The per-concurrency loop in the generator applies only to single-node configs. The relevant multi-node branch builds a single entry whose `conc` is the whole list:

Code block 3: `utils/matrix_logic/generate_sweep_configs.py` lines 359-371, trimmed excerpt.

```
entry = {
    ...
    Fields.RUNNER.value: runner_value,
    Fields.CONC.value: conc_values,          # the entire conc-list
    Fields.MAX_MODEL_LEN.value: isl + osl + 256,
    Fields.EXP_NAME.value: f"{model_code}_{seq_len_str}",  # dsr1_1k1k or dsr1_8k1k
    ...
}
```

So one search-space entry becomes one GitHub Actions matrix config, which the template passes to one Slurm job, which performs one prefill/decode engine launch and then sweeps every concurrency in its list against that single launch. Each matrix entry is its own `sbatch` job.

`exp-name` is only `dsr1_1k1k` or `dsr1_8k1k`. The per-row topology and concurrency detail does not live in `exp-name`. It lives in `RESULT_FILENAME` (see section 7).

## 4.3 Topology fields

Each search-space entry has a `prefill` and a `decode` block:

| Field | Meaning |
|---|---|
| `num-worker` | Worker count for the role. Becomes `xP` (prefill) or `yD` (decode). |
| `tp` | Tensor-parallel degree per worker. Combined with nodes to compute the per-worker TP size. |
| `ep` | Expert-parallel integer. `1` means TP-only. `8` marks a wide-EP role. |
| `dp-attn` | SGLang-style data-parallel attention toggle. `false` for this key. |
| `additional-settings` | Free-form `KEY=VALUE` strings exported before the launcher runs. This is where `PREFILL_NODES`, `DECODE_NODES`, `DECODE_MTP_SIZE`, `PREFILL_DP8EP`, `DECODE_DP8EP`, `PREFILL_BLOCK_SIZE`, and `DECODE_BLOCK_SIZE` are set. |

The node count is authoritative from `additional-settings`: `NUM_NODES = PREFILL_NODES + DECODE_NODES` (`submit.sh`), and `sbatch` is called with `-N` / `-n` equal to that sum. `num-worker` is a separate axis. It becomes `xP` / `yD` and the divisor in the per-worker TP size:

Code block 4: `benchmarks/multi_node/amd_utils/submit.sh` lines 116-119.

```
export PREFILL_TP_SIZE=$(( $PREFILL_NODES * $PREFILL_TP / $PREFILL_WORKERS ))
export DECODE_TP_SIZE=$(( $DECODE_NODES * $DECODE_TP / $DECODE_WORKERS ))
```

In this key every row sets `PREFILL_NODES` equal to the prefill `num-worker` and `DECODE_NODES` equal to the decode `num-worker`, so each worker maps to one node of 8 GPUs and the TP size resolves to 8 for the TP8 rows. See section 6.3 for why this equality matters.

## 4.4 Serving profiles

There are two distinct vLLM profiles for this model, both defined in `models_vllm.yaml` under `DeepSeek-R1-0528`. They are not regex variants of one flag string. They have different block size, batched-token budget, compilation config, and AITER settings.

| Profile | Selected when | Parallelism | Block size |
|---|---|---|---|
| TP8 | `ep: 1`, DP8EP false | One worker spans 8 GPUs with `--tensor-parallel-size 8` | 64 by default; c256 prefill overrides this to 1 |
| DP8EP (wide-EP) | `PREFILL_DP8EP=true` or `DECODE_DP8EP=true` | `--tensor-parallel-size 1 --data-parallel-size 8 --enable-expert-parallel --all2all-backend mori` | 1 |

`server_vllm.sh` chooses the profile per role. For DP8EP it appends the parallelism flags from `GPUS_PER_NODE` and ignores the TP-size injection. For TP8 it rewrites or appends `--tensor-parallel-size` to the computed size.

`server_vllm.sh` then applies optional `PREFILL_BLOCK_SIZE` and `DECODE_BLOCK_SIZE` overrides. The `8k1k c256` row uses this hook so TP8 prefill and DP8EP decode both serve with block size 1.

## 4.5 Speculative decoding (MTP)

`spec-decoding: mtp` marks a row as using DeepSeek MTP speculative decoding. The depth comes from `DECODE_MTP_SIZE` in `additional-settings` and ranges over 0, 1, 2, 3, 4. When the depth is greater than 0, `server_vllm.sh` injects the same speculative config onto both the prefill and decode servers, because the MTP layers must match across the PD pair for KV transfer:

Code block 5: `benchmarks/multi_node/amd_utils/server_vllm.sh` lines 238-242.

```
if [[ "${DECODE_MTP_SIZE:-0}" -gt 0 ]]; then
    _mtp_spec_flag="--speculative-config '{\"method\":\"deepseek_mtp\",\"num_speculative_tokens\":$
{DECODE_MTP_SIZE}}'"
    PREFILL_SERVER_CONFIG+=" ${_mtp_spec_flag}"
    DECODE_SERVER_CONFIG+=" ${_mtp_spec_flag}"
fi
```

The speculative config is not hardcoded in `models_vllm.yaml`. Hardcoding it there would override the per-row depth.

## 4.6 The current matrix

The `dsr1-fp8-mi300x-vllm-disagg` key currently has 9 search-space entries that cover 20 concurrency points. Each entry is one CI job and one engine launch.

| Seq | Concurrencies | Prefill (workers/ nodes) | Decode (workers/ nodes) | Nodes | Profile | Spec / MTP |
|---|---|---|---|---|---|---|
| 1k1k | 1, 6, 9, 30, 60, 117 | 1 / 1 | 1 / 1 | 2 | TP8 (1P1D) | mtp / 3 |
| 1k1k | 231 | 1 / 1 | 1 / 1 | 2 | TP8 (1P1D) | mtp / 1 |
| 1k1k | 462, 615, 1229 | 1 / 1 | 1 / 1 | 2 | DP8EP (1P1D) | mtp / 1 |
| 8k1k | 1 | 1 / 1 | 1 / 1 | 2 | TP8 (1P1D) | mtp / 4 |
| 8k1k | 2 | 1 / 1 | 2 / 2 | 3 | TP8 (1P2D) | mtp / 4 |

| Seq | Concurrencies | Prefill (workers/ nodes) | Decode (workers/ nodes) | Nodes | Profile | Spec / MTP |
|---|---|---|---|---|---|---|
| 8k1k | 6, 9, 16, 24, 30 | 1 / 1 | 3 / 3 | 4 | TP8 (1P3D) | mtp / 3 |
| 8k1k | 77 | 1 / 1 | 1 / 1 | 2 | TP8 (1P1D) | mtp / 3 |
| 8k1k | 154 | 2 / 2 | 2 / 2 | 4 | TP8 (2P2D) | mtp / 3 |
| 8k1k | 256 | 2 / 2 | 1 / 1 | 3 | TP8 prefill, DP8EP decode (2P1D) | none / 0 |

The 20 concurrency points are 1k1k {1, 6, 9, 30, 60, 117, 231, 462, 615, 1229} and 8k1k {1, 2, 6, 9, 16, 24, 30, 77, 154, 256}. The node count shown is per row ( `PREFILL_NODES + DECODE_NODES` ). It is not summed across rows, since each row is a separate job.

There is no 8k1k concurrency-512 row in this revision.

# 5. Key environment variables

Variables are set at one stage and consumed downstream. The chain is workflow env to launcher to recipe to `submit.sh` to `--export=ALL` to `job.slurm` to the Docker `-e` list to `server_vllm.sh` .

| Variable | Set by | Consumed by | Meaning |
|---|---|---|---|
| `RUNNER_NAME` | workflow ( `runner.name` ) | launcher selection, `submit.sh --job-name` | The specific runner and the Slurm job name. |
| `RUNNER_TYPE` | workflow ( `inputs.runner` ) | result processing | The runner type, for example `mi300x-disagg` . |
| `EXP_NAME` | matrix ( `dsr1_8k1k` ) | launcher | Recipe name derivation: `${EXP_NAME%%_*}_${PRECISION}_mi300x_${FRAMEWORK}.sh` . |
| `IMAGE` | matrix | recipe, `submit.sh` , `job.slurm` | Container image, exported as `CONTAINER_IMAGE` then `DOCKER_IMAGE_NAME` . |
| `MODEL` | matrix | launcher | Model id. The basename becomes `MODEL_NAME` and the `models_vllm.yaml` key. |
| `MODEL_PATH` / `MODEL_DIR` | launcher ( `/models/models` default) | `job.slurm` | Host weights directory, mounted to `/models` in the container. |
| `ISL` / `OSL` | matrix | `submit.sh` ( `BENCH_INPUT_LEN` / `BENCH_OUTPUT_LEN` ) | Input and output sequence lengths. |
| `CONC_LIST` | workflow (joined conc-list) | recipe, `bench.sh` | Concurrency sweep. Spaces become `x` before `submit.sh` . |
| `PREFILL_NUM_WORKERS` / `DECODE_NUM_WORKERS` | matrix | `submit.sh` ( `xP` / `yD` ) | Worker counts and node-rank role boundaries. |
| `PREFILL_NODES` / `DECODE_NODES` | `additional-settings` | `submit.sh` ( `NUM_NODES` ) | Allocation size. Their sum is the `sbatch -N` / `-n` . |
| `PREFILL_TP` / `DECODE_TP` | matrix | `submit.sh` ( `*_TP_SIZE` ) | Per-worker tensor-parallel degree input. |
| `PREFILL_EP` / `DECODE_EP` | matrix | recipe ( `*_ENABLE_EP` ) | `1` means TP-only; `8` marks wide-EP. |
| `PREFILL_DP8EP` / `DECODE_DP8EP` | `additional-settings` | `server_vllm.sh` | Selects the DP8EP profile for that role. |
| `DECODE_MTP_SIZE` | `additional-settings` | `server_vllm.sh` | MTP speculative depth, 0 to 4. |
| | `additional-settings` | `job.slurm` , `server_vllm.sh` | |

| Variable | Set by | Consumed by | Meaning |
|---|---|---|---|
| `PREFILL_BLOCK_SIZE` / `DECODE_BLOCK_SIZE` | | | Per-row vLLM `--block-size` override. The `8k1k c256` row sets both roles to 1. |
| `SPEC_DECODING` | matrix | `bench.sh` ( `IS_MTP` ) | `mtp` adds `--use-chat-template` to the benchmark client. |
| `ROUTER_TYPE` | recipe ( `moriio` ) | `job.slurm` | Selects the proxy/router branch. See section 6.4. |
| `RANDOM_RANGE_RATIO` | workflow | `bench.sh` | Prompt length jitter. |
| `RESULT_FILENAME` | workflow | launcher, upload steps | Unique per-row result file prefix. |
| `BENCHMARK_LOGS_DIR` | launcher | `job.slurm` , launcher | NFS log directory the submit host can read. |
| `KEEP_LOGS` | caller (manual path) | launcher | When `1` , the launcher keeps `BENCHMARK_LOGS_DIR` on exit. |
| `VLLM_LOG_ARCHIVE_DIR` | caller | `job.slurm` | When set, per-node vLLM logs are copied to this shared dir before the allocation ends. |
| `IBDEVICES` / `MORI_RDMA_DEVICES` / `MORI_RDMA_TC` | launcher | `env.sh` , `job.slurm` | RDMA device names and traffic class for the MoRIIO KV transport. |

# 6. End-to-end flow

## 6.1 YAML to matrix to jobs

`e2e-tests.yml` checks out the requested ref, runs `generate_sweep_configs.py` , and splits the resulting JSON. Entries that contain a `prefill` key and are not agentic go to the multi-node group:

Code block 6: `.github/workflows/e2e-tests.yml` lines 72-75, trimmed excerpt.

```
MULTI=$(echo "$CONFIG_JSON" | python3 -c "import sys,json; d=json.load(sys.stdin); print(json.dumps([x for x
in d if 'prefill' in x and x.get('scenario-type') != 'agentic-coding' and not x.get('eval-only', False)]))")
```

The `test-sweep-multi-node` job then fans out one `benchmark-multinode-tmpl.yml` call per matrix entry.

## 6.2 Template to runner to recipe

`benchmark-multinode-tmpl.yml` sets `runs-on: ${{ inputs.runner }}` (the runner type), exports the per-row env, expands the prefill and decode `additional-settings` into real exports, then runs the launcher:

Code block 7: `.github/workflows/benchmark-multinode-tmpl.yml` lines 213-215.

```
export ${{ join(fromJson(inputs.prefill-additional-settings), ' ') }} ${{ join(fromJson(inputs.decode-
additional-settings), ' ') }}
export IS_MULTINODE=true
bash ./runners/launch_${RUNNER_NAME%%_*}.sh
```

`launch_mi300x-amds.sh` takes the multi-node branch when `IS_MULTINODE=true` . It sets the Slurm partition ( `compute` ), the host model directory, the RDMA device list, and `GPUS_PER_NODE=8` , then derives and runs the recipe:

Code block 8: `runners/launch_mi300x-amds.sh` lines 80-83, trimmed excerpt.

```
SCRIPT_NAME="${EXP_NAME%%_*}_${PRECISION}_mi300x_${FRAMEWORK}.sh"
JOB_ID=$(bash "benchmarks/${BENCHMARK_SUBDIR}/${SCRIPT_NAME}")
```

For `EXP_NAME=dsr1_8k1k`, `PRECISION=fp8`, `FRAMEWORK=vllm-disagg` this is `benchmarks/multi_node/dsr1_fp8_mi300x_vllm-disagg.sh`.

## 6.3 Recipe to submit to Slurm

The recipe is fully parameterized. It does not branch per topology. It converts the integer `ep` fields into the boolean flags the rest of the chain expects, re-exports the MTP, DP8EP, and router settings so they survive `--export=ALL`, then calls `submit.sh` with a fixed argument order:

Code block 9: `benchmarks/multi_node/dsr1_fp8_mi300x_vllm-disagg.sh` lines 117-126.

```
JOB_ID=$(bash ./submit.sh $PREFILL_NODES \
    $PREFILL_NUM_WORKERS \
    $DECODE_NODES \
    $DECODE_NUM_WORKERS \
    $ISL $OSL "${CONC_LIST// /x}" inf \
    ${PREFILL_ENABLE_EP} ${PREFILL_ENABLE_DP} \
    ${DECODE_ENABLE_EP} ${DECODE_ENABLE_DP} \
    ${PREFILL_TP} ${DECODE_TP} \
    ${RANDOM_RANGE_RATIO} \
    "${NODELIST:-}")
```

Note the first four arguments: prefill nodes, prefill workers, decode nodes, decode workers. `submit.sh` reads them as separate inputs. It computes the allocation from the node arguments (`NUM_NODES = PREFILL_NODES + DECODE_NODES`) and the worker counts from the worker arguments (`xP`, `yD`). `job.slurm` then recomputes `NUM_NODES = xP + yD` from the workers and selects only the first `xP + yD` hostnames of the allocation:

Code block 10: `benchmarks/multi_node/amd_utils/job.slurm` lines 223-228.

```
NUM_NODES=$((xP + yD))
FULL_NODELIST=$(scontrol show hostnames "$SLURM_JOB_NODELIST")
SELECTED_NODES=$(echo "$FULL_NODELIST" | head -n $NUM_NODES)
```

The allocated count and the runtime-used count come from different inputs. They agree only when workers equal nodes for each role. If workers were fewer than nodes, the extra allocated nodes would sit idle. If workers were more than nodes, the selection would truncate and the run would break. Every row in this key keeps them equal.

If a caller passes an explicit `NODELIST`, `submit.sh` rejects it unless its host count equals `NUM_NODES`:

Code block 11: `benchmarks/multi_node/amd_utils/submit.sh` lines 158-164, trimmed excerpt.

```
if [[ "${#NODE_ARR[@]}" -ne "$NUM_NODES" ]]; then
    echo "Error: NODE_LIST has ${#NODE_ARR[@]} nodes but NUM_NODES=${NUM_NODES}" >&2
    exit 1
fi
```

`submit.sh` then submits the batch script with everything exported:

Code block 12: `benchmarks/multi_node/amd_utils/submit.sh` lines 227-243, trimmed excerpt.

```
sbatch_cmd=(
    sbatch --parsable --export=ALL --exclusive
    -N "$NUM_NODES" -n "$NUM_NODES"
    ...
    --partition "$SLURM_PARTITION" --account "$SLURM_ACCOUNT"
    --job-name "$RUNNER_NAME"
    --output "${BENCHMARK_LOGS_DIR}/slurm_job-%j.out"
    --error "${BENCHMARK_LOGS_DIR}/slurm_job-%j.err"
    "$(dirname "$0")/job.slurm"
)
```

`SUBMIT_DRY_RUN=1` stops here. It prints the resolved command and the env that `--export=ALL` would propagate, emits a placeholder job id, and exits without calling `sbatch`. It does not reach `vllm serve`, so it does not show server flags.

## 6.4 Slurm to Docker to vLLM

`job.slurm` runs once per job. It:

1. Picks the models YAML by engine (`models_vllm.yaml` for `vllm-disagg`) and fails if the `MODEL_NAME` key is missing.
2. Resolves and validates the model path. For vLLM it searches known host paths; the error on failure is `FATAL: Model '<name>' not found`.
3. Sets `NUM_NODES = xP + yD`, selects that many hostnames, and rewrites `SLURM_NNODES` to the slice.
4. Builds the Docker env list. `NODE_RANK` is set from the Slurm process id, so each node learns its rank:

Code block 13: `benchmarks/multi_node/amd_utils/job.slurm` lines 390-431, trimmed excerpt.

```
DOCKER_ENV_COMMON=(
    -e NODE_RANK=\$SLURM_PROCID
    -e NODE0_ADDR=\$NODE0_ADDR
    -e xP=\$xP
    -e yD=\$yD
    -e PREFILL_TP_SIZE=\$PREFILL_TP_SIZE
    -e DECODE_TP_SIZE=\$DECODE_TP_SIZE
    -e DECODE_MTP_SIZE=\$DECODE_MTP_SIZE
    -e PREFILL_DP8EP=\$PREFILL_DP8EP
    -e DECODE_DP8EP=\$DECODE_DP8EP
    ...
)
```

1. Fans out one task per selected node with `srun --nodelist=...`, and each task starts the main container, which runs `server.sh`. `server.sh` dispatches to `server_vllm.sh` for this engine.
2. On rank 0 it also starts a separate detached container for the router or proxy.

The router branch depends on `ROUTER_TYPE`. The recipe defaults it to `moriio`:

Code block 14: `benchmarks/multi_node/dsr1_fp8_mi300x_vllm-disagg.sh` line 101.

```
export ROUTER_TYPE="${ROUTER_TYPE:-moriio}"
```

`job.slurm` has its own fallback of `vllm-router`, but the recipe overrides it for this config, so the run uses the in-image MoRIIO toy proxy (`examples/disaggregated/disaggregated_serving/moriio_toy_proxy_server.py`) rather than the Rust `vllm-router`. Either way, rank 0 starts the front-end as a detached container and the rank-0 main process skips `exec` so it can stop the front-end after the benchmark finishes.

## 6.5 vLLM role assignment and KV transfer

`server_vllm.sh` sources `env.sh` for the RDMA and NCCL/RCCL environment, then assigns a role from `NODE_RANK`:

Code block 15: `benchmarks/multi_node/amd_utils/server_vllm.sh` lines 5-8.

```
# Node role assignment (by NODE_RANK):
#   0           -> Proxy/Router + first Prefill node  (kv_producer)
#   1..xP-1     -> Additional Prefill nodes            (kv_producer)
#   xP..xP+yD-1 -> Decode nodes                        (kv_consumer)
```

It assembles the server flags by starting from the `models_vllm.yaml` profile and layering on TP, EP, DP, DP8EP, and MTP. Prefill servers get a `kv_producer` transfer config and decode servers get a `kv_consumer` config, both using `MoRIIOConnector` with the proxy IP and ports nested inside `kv_connector_extra_config`. Each role launches `vllm serve` with its config and writes a per-role log under `/run_logs/slurm_job-<id>/`.

## 6.6 Benchmark and eval

After the servers report healthy, rank 0 runs the benchmark client:

Code block 16: `benchmarks/multi_node/amd_utils/server_vllm.sh` lines 406-409.

```
BENCH_CMD="bash $WS_PATH/bench.sh ${xP} ${yD} $((GPUS_PER_NODE*xP)) $((GPUS_PER_NODE*yD)) \
    $MODEL_DIR $MODEL_NAME /run_logs/slurm_job-${SLURM_JOB_ID} ${BENCH_INPUT_LEN} \
    ${BENCH_OUTPUT_LEN} \"${BENCH_MAX_CONCURRENCY}\" ${BENCH_REQUEST_RATE} \
    ${BENCH_RANDOM_RANGE_RATIO} ${BENCH_NUM_PROMPTS_MULTIPLIER}"
```

`bench.sh` splits the concurrency string on `x` and runs `benchmark_serving.py` once per concurrency, writing one JSON per point (see section 7). When `RUN_EVAL=true`, rank 0 then runs `lm-eval` against the proxy. The eval context window is auto-derived from the served `--max-model-len`, taking the minimum across the prefill and decode configs:

Code block 17: `benchmarks/multi_node/amd_utils/server_vllm.sh` lines 447-452, trimmed excerpt.

```
if [[ -z "${EVAL_MAX_MODEL_LEN:-}" ]]; then
    EVAL_MAX_MODEL_LEN=$(printf '%s\n%s\n' "$PREFILL_SERVER_CONFIG" "$DECODE_SERVER_CONFIG" | grep -oE -- '--
max-model-len[ =]+[0-9]+' | grep -oE '[0-9]+' | sort -n | head -1)
fi
```

# 7. Result and log artifacts

## 7.1 Two meanings of max-model-len

`max-model-len` appears in two unrelated places. The matrix entry carries `max-model-len = isl + osl + 256` (2304 for 1k1k, 9472 for 8k1k) as a generic schema field. The served vLLM window comes from `models_vllm.yaml`: TP8 uses `--max-model-len 32768`, while DP8EP uses `--max-model-len 10240`. Mixed rows use the per-role values, and eval uses the minimum across the served prefill and decode configs.

## 7.2 Per-concurrency result files

`bench.sh` names each result by concurrency, request rate, and GPU split:

Code block 18: `benchmarks/multi_node/amd_utils/bench.sh` line 60.

```
export_file="${profile_folder}/concurrency_${max_concurrency}_req_rate_${chosen_req_rate}_gpus_$
((prefill_gpus+decode_gpus))_ctx_${prefill_gpus}_gen_${decode_gpus}"
```

So a file name encodes the concurrency, the total GPU count, the prefill GPUs (`_ctx_`), and the decode GPUs (`_gen_`). For this recipe the request-rate argument is `inf`; the `req_rate_<r>` field is a filename label, not a separate serving throttle. `process_result.py` reads the GPU fields back out of the file name to compute per-GPU throughput.

## 7.3 Naming and upload

`RESULT_FILENAME` is the unique per-row prefix. It is built in the multi-node template from the experiment name, precision, framework, the full prefill and decode topology, disagg flag, spec, and the concurrency list. The launcher copies each result file from the Slurm log tree to the workspace as `${RESULT_FILENAME}_<file>`. The workflow then runs `process_result.py` per file and uploads artifacts:

| Artifact | Contents |
|---|---|
| `bmk_<RESULT_FILENAME>` | Per-job processed JSON (`agg_<RESULT_FILENAME>_*.json`). |
| `multinode_server_logs_<RESULT_FILENAME>` | `multinode_server_logs.tar.gz` only when the launcher produced it. `launch_mi300x-amds.sh` does not create that tarball, so this artifact is empty on the MI300X path. |
| `results_bmk` | `agg_bmk.json`, aggregated across the run by `collect-results.yml`. |
| `run-stats` | `run_stats.json` from `calc_success_rate.py`. |

### 7.4 Log lifecycle

Per-node vLLM logs live on each node under `/tmp` (mounted as `/run_logs`), as `prefill_<host>.log`, `decode_<host>.log`, and `moriio_proxy_<host>.log`. They are local to the node, not on shared storage. Node 0 copies `/run_logs/slurm_job-<id>` into `BENCHMARK_LOGS_DIR/logs`, and the launcher wipes `BENCHMARK_LOGS_DIR/logs` after copying result metrics, so raw server logs do not survive there by default.

The `multinode_server_logs_<RESULT_FILENAME>` upload exists in the shared template but depends on a `multinode_server_logs.tar.gz` that only some launchers produce. `launch_mi300x-amds.sh` does not produce it, so on the MI300X path that artifact is empty. To preserve raw per-node logs for this path, set `VLLM_LOG_ARCHIVE_DIR`; `job.slurm` then copies each node's `/tmp/slurm_job-<id>` to that shared directory while the allocation is still held, before the launcher cleanup runs. The launcher also copies the Slurm `.out` and `.err` files to `benchmark_artifacts`, but this template does not upload that path. Result metric JSON survives because the launcher copies it to the workspace.

## 8. Safe edit rules

These rules keep edits consistent with how the chain reads the config.

Change topology and concurrency in `amd-master.yaml`, not in the scripts. The recipe is generic and has no per-row branches. Add or change a row by editing the `search-space` of the `dsr1-fp8-mi300x-vllm-disagg` key.

Keep `PREFILL_NODES` equal to the prefill `num-worker` and `DECODE_NODES` equal to the decode `num-worker` in `additional-settings`. The allocation uses the node values and the runtime uses the worker values, and `server_vllm.sh` assigns one role per node. If they diverge the run either wastes nodes or breaks.

Set the per-worker degree through `tp`, `ep`, and the DP8EP flags, not by hand-editing `--tensor-parallel-size`. `server_vllm.sh` computes and injects the TP size, and the DP8EP branch replaces it entirely.

Change vLLM model flags in `models_vllm.yaml` under `DeepSeek-R1-0528`, not in the recipe or `server_vllm.sh`. Keep the TP8 profile and the DP8EP profile separate. They are different policies, not one with substitutions.

Do not hardcode `--speculative-config` in `models_vllm.yaml`. MTP depth comes from `DECODE_MTP_SIZE` per row, and `server_vllm.sh` applies the same config to both PD roles.

Keep the served `--max-model-len` in `models_vllm.yaml`. The matrix `max-model-len` field is a generic schema value and does not size the vLLM window for this model.

Add a new model by adding a top-level entry to `models_vllm.yaml` and a new key to `amd-master.yaml`. No script change is needed for a model that fits the existing schema.

Treat `ROUTER_TYPE` as part of the recipe. The recipe defaults it to `moriio` for this config. `job.slurm` has its own `vllm-router` fallback for other paths, so do not assume one global default.

Do not name specific compute nodes in the config or scripts. Node selection is dynamic. Pin nodes only through a validated `NODELIST` whose count matches `NUM_NODES`.

Map a runner type to runners in `runners.yaml`. The `runner:` value in `amd-master.yaml` is a type. The set of machines that can serve it lives in `runners.yaml`.